The Categorical Abstract Machine

A Tutorial Introduction

November 3, 2026

Abstract

The Categorical Abstract Machine (CAM) of Cousineau, Curien and Mauny is a small and elegant machine for the call-by-value λ-calculus. It gave the Caml language its name. Its origins lie in category theory, but the machine itself needs none: it is what you get from the SECD machine if you replace variable names by positions, represent environments as nested pairs, and compile programs ahead of time. Three registers are then enough, and the whole instruction set fits on half a page. This tutorial develops the machine from those ideas, gives its transition rules and its compiler, and traces several programs step by step, including a recursive factorial. It then explains, without assuming any category theory, where the name comes from: the compiled code is a composition of a handful of functions on pairs, and the laws those functions obey are the laws of a cartesian closed category. A final part looks at a complete implementation on the VM16 virtual machine, with a copying garbage collector, and measures it against the SECD machine and against hand-written code. The prerequisites are the untyped λ-calculus and, ideally, the companion tutorial on the SECD machine.

Contents

1 Introduction **2**

2 From the SECD machine to the CAM **3**
2.1 Environments and closures, briefly . . . 3
2.2 From names to positions . . . 3
2.3 Environments as nested pairs . . . 3
2.4 One register for S and E . . . 4

3 The machine **4**
3.1 Values and states . . . 4
3.2 Instructions . . . 5

4 Compiling for the CAM **6**
4.1 The compiler . . . 6
4.2 The pairing dance . . . 7
4.3 An example . . . 7

5 Worked examples **8**
5.1 A function call . . . 8
5.2 Two arguments: $K\,4\,3$ . . . 8

6 Conditionals and recursion **9**
6.1 A conditional that evaluates one branch . . . 9
6.2 Recursion . . . 10
6.3 Factorial . . . 10

7 Where the name comes from **11**
7.1 Code as functions . . . 11
7.2 The laws . . . 12
7.3 Why the reading pays off . . . 12

8 An implementation on VM16 **13**
8.1 Representation . . . 13
8.2 The interpreter . . . 13
8.3 Garbage collection . . . 14

9 Programs with definitions **15**

10 Measurements **16**

11 Variations and descendants **17**

12 Summary **18**

Exercises **18**

13 Reference card **19**

1 Introduction

In 1964 Peter Landin showed how to evaluate λ-terms with a machine [13]. His SECD machine keeps four lists: a stack S of values, an environment E that maps names to values, a control C of expressions still to be evaluated, and a dump D of suspended calls. Twenty years later, Guy Cousineau, Pierre-Louis Curien and Michel Mauny proposed a different machine [3, 4]. It came out of Curien's work on *categorical combinators* [5], a way of translating λ-terms into a few combinators taken from category theory. Running those combinators directly gave the *Categorical Abstract Machine*.

The CAM turned out to be very practical. It was the target of the first implementation of ML at INRIA, and the language took its name from it: Caml is the *Categorical Abstract Machine Language*. Later Caml systems moved to other machines [14], but the CAM remains one of the clearest examples of how to compile a functional language.

This tutorial approaches the CAM from the SECD side. We will see that three design changes turn Landin's machine into the CAM:

1. **Positions instead of names.** A variable is replaced by its distance from its binder, its *de Bruijn index*, so that the machine never searches for a name.
2. **Environments are values.** An environment is a chain of nested pairs, so it can sit in the same register as any other value. The SECD machine's S and E collapse into a single register, the *term*.
3. **Compile first.** The program is translated once into a sequence of instructions, so there is no expression tree to take apart at run time. Return addresses go on the stack, and the dump disappears.

Section 2 develops these ideas, and section 3 gives the machine. Section 4 shows how to compile to it, and sections 5 and 6 run programs, ending with a factorial. Section 7 explains the categorical reading, and sections 8 and 10 present and measure a complete implementation. A reference card, section 13, collects the rules on one page.

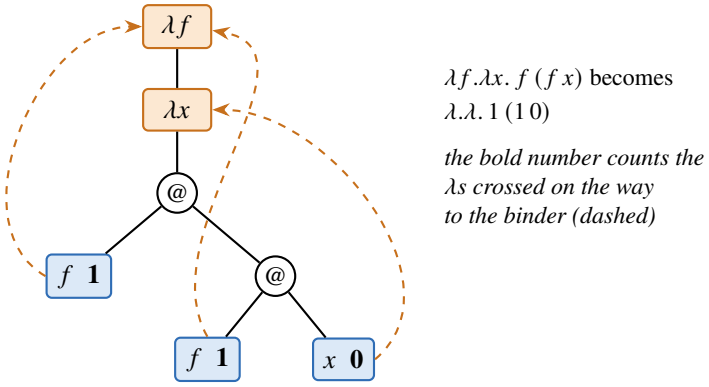


Figure 1: De Bruijn indices for the Church numeral 2. Each variable is replaced by the number of λs between it and its binder.

What you need to know

We assume the untyped λ-calculus: terms x, $\lambda x.M$ and $M\,N$, free and bound variables, and β-reduction. We use call-by-value: a function's argument is evaluated before the call, and evaluation stops at a λ. The companion tutorial *The SECD Machine* covers environments and closures in detail; we review them briefly here. No category theory is needed anywhere.

The examples use the input language of the implementation in section 8: single-letter variables, `\x y.M` for $\lambda x.\lambda y.M$, integers, the curried operators `+ - * / % = <`, a conditional `if c t e`, and recursive functions `rec f.\x.M`.

2 From the SECD machine to the CAM

2.1 Environments and closures, briefly

An abstract machine for the λ-calculus avoids substitution. Instead of replacing x by a value in the body of $\lambda x.M$, it evaluates M in an *environment* that records the value of x. A function value must then remember the environment it was created in, so that its free variables keep the meaning they had there. The pair of code and environment is a *closure*. Both ideas carry over unchanged to the CAM; what changes is how environments are represented.

2.2 From names to positions

In the SECD machine, the environment is a list of bindings $[x \mapsto 4,\ y \mapsto 3,\ \ldots]$, and every variable occurrence is looked up by comparing names. That is wasted work: the shape of the environment at any point of a program is known before the program runs. In $\lambda a.\lambda b.a$, the body a always runs in an environment whose newest binding is b and whose next binding is a. So a can be compiled to "the second binding", and the name can be thrown away.

Definition 1. The *de Bruijn index* of a variable occurrence is the number of λs between the occurrence and its binder [7]. Index 0 refers to the innermost enclosing λ.

Figure 1 shows the indices for a small term. The same variable can have different indices at different occurrences, because the count depends on where the occurrence is.

2.3 Environments as nested pairs

If variables are positions, an environment no longer needs names; it only needs to hold values in order. The CAM uses the simplest structure that does this, a chain of pairs. The empty environment is the empty

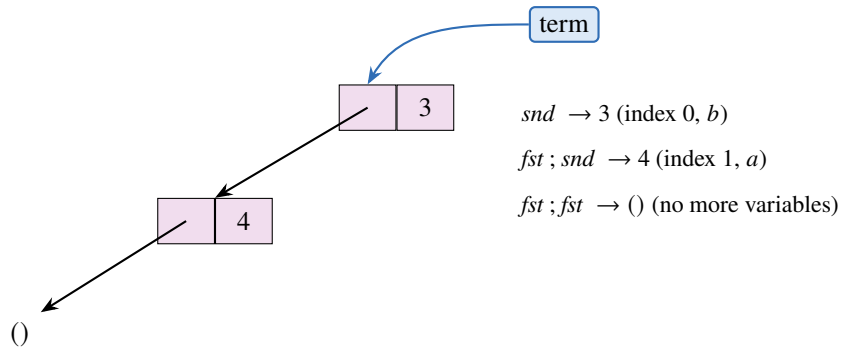


Figure 2: The environment (((), 4), 3) as heap cells. Each pair has a first field (left half) pointing to the older environment and a second field (right half) holding a value. Variables are reached by following first fields.

tuple (), and binding a new value v in environment e gives the pair $(e,\ v)$. Inside $\lambda a.\lambda b.\ldots$, applied to 4 and 3, the environment is

$$(((),\ 4),\ 3).$$

The newest value is always the second component of the outermost pair, the next one is one step further in, and so on. The variable with index i is found by taking the first component i times and then the second component once (fig. 2):

$$\text{index } i \quad \longmapsto \quad \underbrace{fst\ ;\cdots;\ fst}_{i}\ ;snd.$$

Two things follow. First, extending an environment is a single allocation, and the old environment is shared, not copied. Second, an environment is an ordinary value. Pairs are values in the CAM, so a single register can hold either an environment or the result of a computation, and the instructions that save and restore one also work for the other.

2.4 One register for S and E

The SECD machine keeps the current environment in E and intermediate values on S. The CAM merges the two into one register, the *term*. At the start of evaluating an expression the term holds the environment, and at the end it holds the expression's value. In between, environments that will be needed again are saved on the stack.

The dump goes as well. Because the CAM runs compiled code, a pending call only needs to remember where to continue, a single code address, and that address fits on the same stack. Figure 3 compares the two machines.

3 The machine

3.1 Values and states

The CAM manipulates four kinds of value:

- numbers n;
- the empty tuple (), the empty environment;
- pairs $(s,\ t)$ of values;
- closures $[C : s]$, a piece of code C together with the environment s it was created in.

A state is a triple $\langle t,\ C,\ S\rangle$: the term t, the code C still to run, and the stack S. We write $c\ ;\ C$ for code starting with instruction c, and $v.S$ for a stack with v on top. The stack holds values and also code

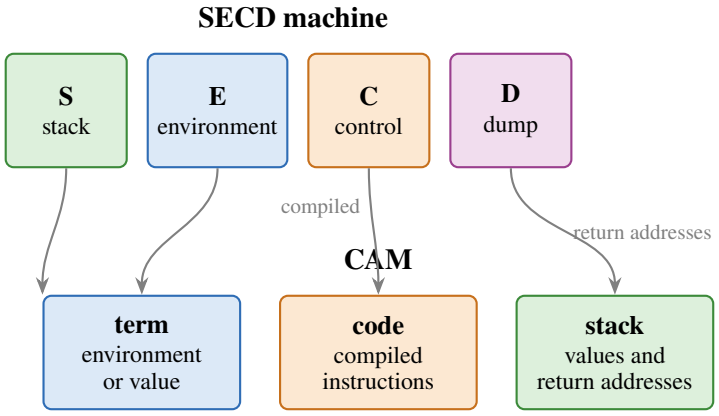


Figure 3: The SECD machine's four registers and the CAM's three. The term takes over the roles of S and E, compiled code replaces the control, and return addresses on the stack replace the dump. The stack also keeps the intermediate values that S used to hold.

	$\langle \text{term}, \text{code}, \text{stack} \rangle \longrightarrow \langle \text{term}, \text{code}, \text{stack} \rangle$
ACC i	$\langle t, \mathsf{ACC}\, i ; C, S \rangle \longrightarrow \langle snd(fst^i(t)), C, S \rangle$
QUOTE n	$\langle t, \mathsf{QUOTE}\, n ; C, S \rangle \longrightarrow \langle n, C, S \rangle$
PUSH	$\langle t, \mathsf{PUSH} ; C, S \rangle \longrightarrow \langle t, C, t.S \rangle$
SWAP	$\langle t, \mathsf{SWAP} ; C, s.S \rangle \longrightarrow \langle s, C, t.S \rangle$
CONS	$\langle t, \mathsf{CONS} ; C, s.S \rangle \longrightarrow \langle (s, t), C, S \rangle$
CUR C'	$\langle t, \mathsf{CUR}\, C' ; C, S \rangle \longrightarrow \langle [C' : t], C, S \rangle$
APP	$\langle ([C' : s], t), \mathsf{APP} ; C, S \rangle \longrightarrow \langle (s, t), C', C.S \rangle$
RET	$\langle t, \mathsf{RET} ; C, C'.S \rangle \longrightarrow \langle t, C', S \rangle$
op	$\langle n_2, op ; C, n_1.S \rangle \longrightarrow \langle n_1\ op\ n_2, C, S \rangle$
BRANCH $C_1\ C_2$	$\langle b, \mathsf{BRANCH}\, C_1\, C_2 ; C, s.S \rangle \longrightarrow \langle s, C_b ; C, S \rangle \quad C_b = C_1$ if $b \neq 0$, else C_2
REC C'	$\langle t, \mathsf{REC}\, C' ; C, S \rangle \longrightarrow \langle k, C, S \rangle \quad$ where $k = [C' : (t, k)]$

Figure 4: The transition rules of the CAM. ACC, QUOTE load the term; PUSH, SWAP, CONS manage the stack; CUR, APP, RET create and call functions; the last group adds arithmetic, conditionals and recursion.

sequences, which act as return addresses. A program P starts in the state $\langle (), P, \varepsilon \rangle$, with the empty environment and an empty stack, and stops when it reaches the instruction STOP; the term is then the result.

3.2 Instructions

Figure 4 gives the whole machine. There are four groups.

Loading the term.
ACC i replaces the environment in the term by the value of variable i. The original CAM has separate instructions FST and SND; ACC i abbreviates i FSTs and one SND. QUOTE n replaces the term by a constant.

The stack.
PUSH saves a copy of the term. SWAP exchanges the term with the top of the stack: typically it saves a result and brings back a saved environment. CONS pairs the top of the stack with the term. These three are all the CAM needs to evaluate two expressions in the same environment and combine their values (section 4.2).

Functions.
CUR C' makes a closure of the code C' and the current environment; its name comes from *currying*. APP expects a pair of a closure $[C' : s]$ and an argument t. It builds the closure's environment extended with the argument, (s, t), pushes the rest of the code as a return address, and jumps to C'. When C' finishes, RET pops the return address.

Extensions.
Arithmetic operators take their left operand from the stack and their right operand from the term. BRANCH tests the term, restores a saved environment, and continues with one of two code sequences. REC builds a closure that contains itself, for recursive functions. None of these is part of the categorical core; section 6 discusses them.

Notice how little each instruction does. There is no rule that walks a list of bindings, no rule that takes an expression tree apart, and no rule that saves three registers at once. Every transition does a constant amount of work except ACC i, which takes $i + 1$ steps, and only CUR, CONS, APP and REC allocate memory.

4 Compiling for the CAM

4.1 The compiler

The compiler translates a term M into code $[\![M]\!]_\rho$, where ρ is the list of variable names in scope, innermost first. The position of a name in ρ is its de Bruijn index:

$$
\begin{aligned}
[\![x]\!]_\rho &= \mathsf{ACC}\ i \qquad \text{where } x \text{ is at position } i \text{ in } \rho \\
[\![n]\!]_\rho &= \mathsf{QUOTE}\ n \\
[\![\lambda x.M]\!]_\rho &= \mathsf{CUR}\ ([\![M]\!]_{x:\rho}\ ;\ \mathsf{RET}) \\
[\![M\ N]\!]_\rho &= \mathsf{PUSH}\ ;\ [\![M]\!]_\rho\ ;\ \mathsf{SWAP}\ ;\ [\![N]\!]_\rho\ ;\ \mathsf{CONS}\ ;\ \mathsf{APP}
\end{aligned}
$$

That is the whole compiler for the pure λ-calculus. The extensions add

$$
\begin{aligned}
[\![op\ M\ N]\!]_\rho &= \mathsf{PUSH}\ ;\ [\![M]\!]_\rho\ ;\ \mathsf{SWAP}\ ;\ [\![N]\!]_\rho\ ;\ op \\
[\![\mathbf{if}\ c\ t\ e]\!]_\rho &= \mathsf{PUSH}\ ;\ [\![c]\!]_\rho\ ;\ \mathsf{BRANCH}\ [\![t]\!]_\rho\ [\![e]\!]_\rho \\
[\![\mathbf{rec}\ f.\lambda x.M]\!]_\rho &= \mathsf{REC}\ ([\![M]\!]_{x:f:\rho}\ ;\ \mathsf{RET})
\end{aligned}
$$

and an operator used on its own, such as `(+ 1)`, is first expanded to $\lambda a.\lambda b.\ op\ a\ b$.

The compiler has one invariant that makes everything fit together:

Theorem 1 (Code leaves the stack as it found it)**.** *If M evaluates to the value v in environment e, then*

$$\langle e,\ [\![M]\!]_\rho\ ;\ C,\ S\rangle \longrightarrow {}^*\langle v,\ C,\ S\rangle,$$

where ρ lists the names whose values e holds.

Proof sketch. By induction on M. For a variable, ACC i walks e to the value. For $\lambda x.M$, CUR turns e into the closure, which is a value. For $M\ N$ we follow the code. PUSH saves e. By induction $[\![M]\!]$ turns e into a closure $[C' : s]$ without disturbing the stack, so SWAP leaves e in the term and the closure on the stack. By induction again $[\![N]\!]$ turns e into the argument a, CONS builds $([C' : s],\ a)$, and APP runs C' on $(s,\ a)$ with C pushed. C' is $[\![B]\!]_{y:\rho'}$; RET for the function's body B, so by induction it ends with the result in the term and C on top of the stack, and RET continues with C and S. □

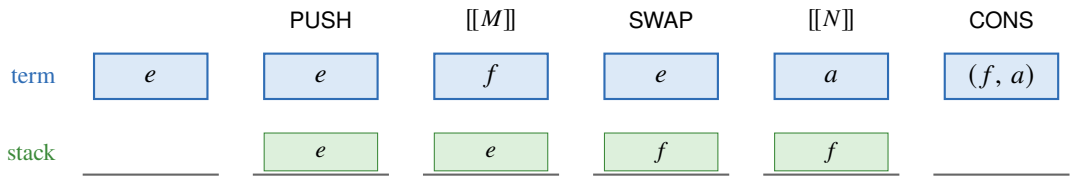


Figure 5: Evaluating M and N in the same environment e. PUSH saves e; $[[M]]$ computes the function f; SWAP parks f and restores e; $[[N]]$ computes the argument a; CONS pairs them. The stack below the line is the same before and after.

4.2 The pairing dance

The heart of the compiler is the application rule. To evaluate $M\ N$, both M and N must be evaluated *in the same environment*, but evaluating M replaces the environment in the term by M's value. PUSH, SWAP and CONS solve this by parking values on the stack, as fig. 5 shows. The same pattern appears every time two values have to be computed from one environment: for applications, for operators, and in the categorical reading of section 7, for pairing in general.

4.3 An example

Compiling $(\lambda x.+\ x\ 1)\ 41$ in the empty environment gives

```
PUSH
CUR  (PUSH; ACC 0; SWAP; QUOTE 1; ADD; RET)    % \x. + x 1
SWAP
QUOTE 41
CONS
APP
STOP
```

The outer PUSH. . . APP is the application rule, with $[[\lambda x.+\ x\ 1]]$ and $[[41]]$ in the two slots. Inside the closure, the operator rule computes $x + 1$, where x is ACC 0 because it is the innermost (and only) variable. STOP is added after the whole program.

In memory, code is a flat sequence of bytes, so the nested code of a CUR is laid out inline, preceded by the address where the code continues. The implementation's disassembler prints the example as

```
3000  PUSH
3001  CUR 300D          % the closure's code is 3004..300C
3004    PUSH
3005    ACC 0
3007    SWAP
3008    QUOTE 1
300B    ADD
300C    RET
300D  SWAP
300E  QUOTE 41
3011  CONS
3012  APP
3013  STOP
```

Thirteen instructions, seven of them at the top level. The next section runs it.

	term	stack	next
0	()	ε	PUSH
1	()	()	CUR B
2	$[B : ()]$	()	SWAP
3	()	$[B : ()]$	QUOTE 41
4	41	$[B : ()]$	CONS
5	$([B : ()], 41)$	ε	APP
6	$((), 41)$	r	PUSH
7	$((), 41)$	$((), 41).r$	ACC 0
8	41	$((), 41).r$	SWAP
9	$((), 41)$	$41.r$	QUOTE 1
10	1	$41.r$	ADD
11	42	r	RET
12	42	ε	STOP

Table 1: The CAM evaluating $(\lambda x.+\ x\ 1)\ 41$. B is the code PUSH; ACC 0; SWAP; QUOTE 1; ADD; RET, and r the return address of STOP.

5 Worked examples

The traces in this section were produced by replaying the code that the implementation of section 8 compiled, and they agree with it step for step. Each row shows the state *before* the instruction in the last column is executed. Return addresses are written r, and closures are written with the label of their code.

5.1 A function call

Table 1 traces $(\lambda x.+\ x\ 1)\ 41$; let B be the code of the closure. Rows 0–5 build the pair of function and argument. Row 5 calls it: APP replaces $([B : ()], 41)$ by the environment $((), 41)$, pushes the return address, and jumps to B. In the body, rows 6–10 are the operator rule, with x fetched by ACC 0, and RET goes back to STOP.

Compare this with the SECD machine. There, the same call creates a dump frame holding three registers, and the body runs with a fresh stack. Here the call pushes one word, and the body uses the caller's stack, above the return address.

5.2 Two arguments: $K\ 4\ 3$

The combinator $K = \lambda a.\lambda b.a$ shows how curried functions and closures interact. Its body a has index 1, so $[[K]] = \mathsf{CUR}\, K_1$ with

$$K_1 = \mathsf{CUR}\ K_2\ ;\ \mathsf{RET}, \qquad K_2 = \mathsf{ACC}\ 1\ ;\ \mathsf{RET}.$$

Table 2 traces $K\ 4\ 3$, which is $(K\ 4)\ 3$. The first APP (row 6) calls K_1 with the environment $((), 4)$. K_1 does no more than wrap that environment into a new closure $[K_2 : ((), 4)]$ and return it (rows 7–8): this closure *is* the partial application $K\ 4$, and the 4 is stored in its environment. The second APP (row 12) extends that environment with 3, so K_2 runs in $(((), 4), 3)$, exactly the environment of fig. 2, and ACC 1 finds the 4.

Figure 6 shows the heap around the second call. The new environment $(s, 3)$ points to the closure's environment s instead of copying it, so any number of partial applications can share it.

	term	stack	next
0	()	ε	PUSH
1	()	()	PUSH
2	()	().()	CUR K_1
3	$[K_1 : ()]$	().()	SWAP
4	()	$[K_1 : ()].()$	QUOTE 4
5	4	$[K_1 : ()].()$	CONS
6	$([K_1 : ()], 4)$	()	APP
7	$((), 4)$	$r_1.()$	CUR K_2
8	$[K_2 : ((), 4)]$	$r_1.()$	RET
9	$[K_2 : ((), 4)]$	()	SWAP
10	()	$[K_2 : ((), 4)]$	QUOTE 3
11	3	$[K_2 : ((), 4)]$	CONS
12	$([K_2 : ((), 4)], 3)$	ε	APP
13	$(((), 4), 3)$	r_2	ACC 1
14	4	r_2	RET
15	4	ε	STOP

Table 2: The CAM evaluating $K\,4\,3$. The closure created at row 8 is the partial application $K\,4$.

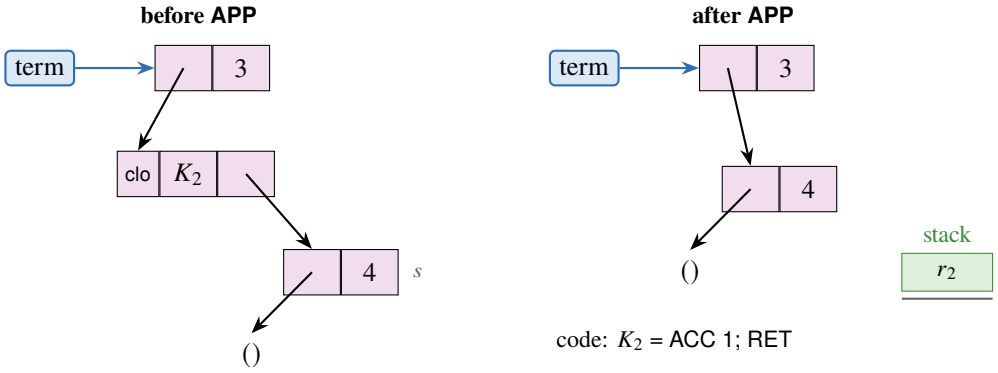


Figure 6: The second call in $K\,4\,3$. Before: the term is the pair of the closure $[K_2 : s]$ and the argument 3. After: the term is $(s, 3)$, which shares s with the closure, the return address is on the stack, and the code is the closure's.

6 Conditionals and recursion

6.1 A conditional that evaluates one branch

With call-by-value, a conditional cannot be an ordinary function: in *if c t e*, both t and e would be evaluated before the call. For Church booleans that is fatal in recursive definitions, because the recursive branch is evaluated even when the base case applies. The SECD tutorial works around this by wrapping both branches in a $\lambda d.$, and Landin did the same in 1964. The CAM has the same problem with Church booleans, since it is a call-by-value machine too: a factorial written with Church booleans and without the wrappers recurses until memory runs out.

The cure is to make the conditional part of the compiler. The rule

$$[[\textbf{if } c\ t\ e]]_\rho = \mathsf{PUSH} \,;\, [[c]]_\rho \,;\, \mathsf{BRANCH}\ [[t]]_\rho\ [[e]]_\rho$$

saves the environment, evaluates the condition, and then BRANCH restores the environment and runs only one of the two code sequences. In memory, BRANCH carries the address of the else-code, and the then-code ends with a JUMP past it:

$$\mathsf{PUSH} \,;\, [[c]] \,;\, \mathsf{BRANCH}\ \mathit{else} \,;\, [[t]] \,;\, \mathsf{JUMP}\ \mathit{end} \,;\ \mathit{else}\colon\ [[e]] \,;\ \mathit{end}\colon$$

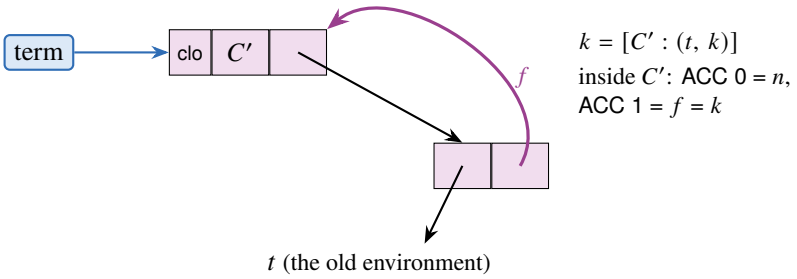


Figure 7: The closure built by REC. Its environment is a pair whose second field points back to the closure, so the function can find itself as variable 1.

This is the CAM's counterpart of the SEL and JOIN instructions of the compiled SECD machine [9], but it needs no join: when the chosen branch finishes, the machine simply continues with the next instruction.

6.2 Recursion

The categorical combinators have no recursion. A recursive function can be written with a fixpoint combinator; the call-by-value one is

$$Z = \lambda f.\,(\lambda x.\,f\,(\lambda y.\,x\,x\,y))\,(\lambda x.\,f\,(\lambda y.\,x\,x\,y)),$$

and it works on the CAM unchanged. But it is slow: each recursive call goes through two extra closures. Real implementations, including Caml's `let rec`, build a closure that refers to itself instead:

$$\langle t,\ \mathsf{REC}\,C'\,;C,\ S\rangle \longrightarrow \langle k,\ C,\ S\rangle \qquad \text{where } k = [C' : (t,\ k)].$$

The closure's environment is the current one, extended with the closure itself. When k is called with argument a, its code runs in $((t,\ k),\ a)$: the argument has index 0 and the function, under the name f, has index 1. The cycle is created by allocating both cells and then filling in their fields (fig. 7).

6.3 Factorial

Here is the factorial function, applied to 7:

```
(rec f.\n. if (= n 0) 1 (* n (f (- n 1)))) 7
```

The compiler produces the code below; the indentation marks the code of the REC closure. Inside it, n is ACC 0 and f is ACC 1.

```
3000  PUSH
3001  REC 302A
3004    PUSH                    % if
3005    PUSH                    %   = n 0
3006    ACC 0
3008    SWAP
3009    QUOTE 0
300C    EQ
300D    BRANCH 3016
3010    QUOTE 1                 % then 1
3013    JUMP 3029
3016    PUSH                    % else * n (f (- n 1))
3017    ACC 0                   %   n
3019    SWAP
301A    PUSH                    %   f (- n 1)
```

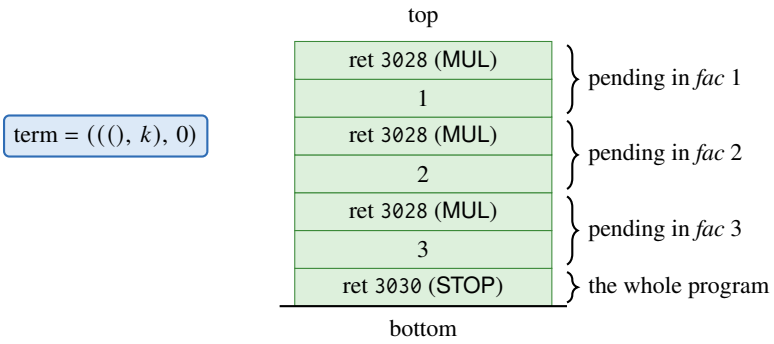


Figure 8: The CAM stack in *fac* 3, as the innermost call $f\ 0$ starts. Each pending multiplication holds its left operand and a return address; nothing else survives the call. At the same point, the SECD machine holds three dump frames of three heap cells each.

```
301B    ACC 1                   %     f
301D    SWAP
301E    PUSH                    %     - n 1
301F    ACC 0
3021    SWAP
3022    QUOTE 1
3025    SUB
3026    CONS
3027    APP
3028    MUL
3029    RET
302A  SWAP                      % apply it to 7
302B  QUOTE 7
302E  CONS
302F  APP
3030  STOP
```

Reading the code against the compilation rules is a good exercise. The $\star$ needs n on the stack while the recursive call runs, so it is computed first and parked by the SWAP at `3019`. The recursive call is an ordinary PUSH… CONS; APP, and when it returns, MUL multiplies the parked n by the result.

The run takes 171 instructions. Figure 8 shows the stack at the deepest point of the smaller computation *fac* 3, when the call $f\ 0$ has just started. Each pending call has left two words: the n waiting to be multiplied and the return address `3028`, the MUL. The environments of the pending calls are not on the stack at all. They are only needed again if a pending call still has to look up a variable, and after the recursive call this function only multiplies, so they are garbage already.

7 Where the name comes from

So far the CAM has been an engineering design. This section explains its mathematical origin, without category theory. The idea is to read every piece of compiled code as a *function*.

7.1 Code as functions

By theorem 1, the code $[\![M]\!]_\rho$ turns an environment in the term into the value of M, and leaves the stack alone. So we can think of it as a function from environments to values. Let us name a few functions on values:

combinator	definition	instructions
Fst	$\mathrm{Fst}(a,\ b) = a$	FST
Snd	$\mathrm{Snd}(a,\ b) = b$	SND
$f\ ;\ g$	$(f\ ;\ g)(x) = g(f(x))$, first f then g	$f\ ;\ g$
$\langle f, g\rangle$	$\langle f, g\rangle(x) = (f(x),\ g(x))$	PUSH;f;SWAP;g;CONS
$\Lambda(f)$	$\Lambda(f)(x) =$ the function $y \mapsto f(x,\ y)$	CUR f
App	$\mathrm{App}(h,\ y) = h(y)$	APP
$'n$	$'n(x) = n$, a constant	QUOTE n

With these, the compiler can be written without any instructions:

$$[[x_i]] = \underbrace{\mathrm{Fst}\ ;\ \cdots\ ;\ \mathrm{Fst}}_{i}\ ;\mathrm{Snd}, \qquad [[\lambda x.M]] = \Lambda([[M]]), \qquad [[M\ N]] = \langle [[M]], [[N]]\rangle\ ;\ \mathrm{App}.$$

A variable is a projection out of the environment. A λ is a function that waits for the rest of its environment. An application computes the function and the argument from the same environment, then applies one to the other. These are Curien's *categorical combinators*, and the third column shows that the CAM is simply a machine that executes them: a sequence of instructions is a composition, read left to right, and $\langle f, g\rangle$ is implemented by the pairing dance of fig. 5. The original paper even writes PUSH, SWAP and CONS as the symbols $\langle$, the comma, and $\rangle$ of $\langle f, g\rangle$.

7.2 The laws

The combinators obey a handful of equations, which follow directly from their definitions:

$$\langle f, g\rangle\ ;\ \mathrm{Fst} = f \qquad \langle f, g\rangle\ ;\ \mathrm{Snd} = g \qquad h\ ;\ \langle f, g\rangle = \langle h\ ;\ f, h\ ;\ g\rangle$$
$$\langle h\ ;\ \Lambda(f), g\rangle\ ;\ \mathrm{App} = \langle h, g\rangle\ ;\ f \qquad (f\ ;\ g)\ ;\ h = f\ ;\ (g\ ;\ h) \qquad \langle \mathrm{Fst}, \mathrm{Snd}\rangle = id$$

The first two say that projections undo pairing. The fourth is β-reduction in disguise: building a closure of f with environment $h(x)$ and applying it to $g(x)$ is the same as running f on $(h(x),\ g(x))$. That is exactly what APP does.

What a category theorist would add

A *category* is any collection of things that compose associatively and have identities, like our functions. One with pairing and projections obeying the first three laws is said to have *products*, and one that also has Λ and App obeying the fourth law is *cartesian closed*. A classic result, due to Lambek, is that the simply typed λ-calculus and cartesian closed categories describe the same thing [12]: every λ-term becomes a combination of these combinators, and every equation between λ-terms becomes an equation between combinations. The CAM takes this correspondence literally and builds a machine whose instructions are the operations of a cartesian closed category. For this tutorial, the table and the laws above are all we need. The untyped calculus we run has no types, but the combinators work the same way.

7.3 Why the reading pays off

Thinking of code as functions has practical consequences. The laws are program transformations that a compiler may apply. For instance, the law $\langle f, g\rangle$;Fst $= f$ says that the code PUSH;f;SWAP;g;CONS;FST can be replaced by f, provided g terminates without side effects. The same view led to a line of research on *explicit substitutions*, calculi in which environments are part of the term language and which explain all the classical abstract machines as strategies for applying substitutions [1].

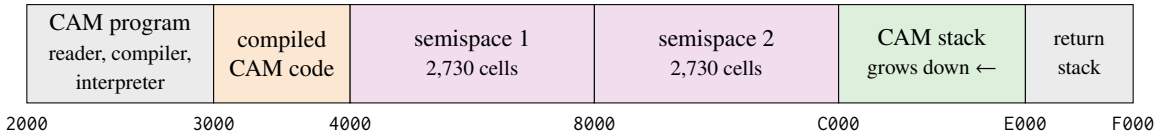


Figure 9: Memory layout of `cam.asm`, not to scale. The compiler writes code from `0x3000`. The heap is two 16 KB semispaces, and the CAM stack grows down from `0xE000` towards the heap. APP stops the program with "out of stack" before the two meet.

8 An implementation on VM16

The file `CAM/cam.asm` implements the machine on VM16, a small 16-bit virtual machine with sixteen registers, a data stack and a return stack, designed to be implemented in a few hundred lines of assembly on 8-bit processors. One VM16 program, about 3 KB, contains a reader for the input syntax, the compiler of section 4, the CAM interpreter, and a garbage collector. It runs unchanged on the portable C interpreter and on a Z80 interpreter.

8.1 Representation

The term and the code pointer live in VM16 registers (`r8` and `r9`). The CAM stack is the VM16 data stack, so PUSH and POP are single VM16 instructions. Values are 16-bit words:

- a number n is stored as $2n + 1$, so numbers are odd, need no allocation, and range over -16384 to 16383;
- () is 0;
- pairs and closures are 3-word heap cells: a tag, then the two fields (fst and snd, or code address and environment).

Tagging makes some arithmetic slightly unusual: $(2a + 1) + (2b + 1) - 1 = 2(a + b) + 1$, so ADD is one VM16 addition and one decrement, and comparisons work on the tagged values directly because tagging preserves the order. Figure 9 shows the memory layout.

8.2 The interpreter

Each CAM instruction is one opcode byte followed by its operands. The main loop counts the instruction, fetches the opcode, and jumps through a table:

```
next:   ADDI    r12, 1                  ; count the instruction
        BZ      r12, count_hi
nx_go:  LB      r0, [r9]                ; fetch the opcode
        ADDI    r9, 1
        SHL     r0
        LI      r1, op_table
        ADD     r1, r0
        LW      r1, [r1]
        JR      r1                      ; jump to its handler
```

Most handlers are a few instructions long. PUSH is `PUSH r8` followed by a jump back to `next`. The longest is APP, which checks that it really has a closure and then builds the new environment. It also checks the depth of the stack: the stack can only grow without bound in a recursion, and every recursion passes through APP.

```
op_app: GETDSP  r1                      ; stop if the stack has grown
        BLTU    r1, r13, app_deep       ; too close to the heap
        CALL    reserve                 ; room for one cell
```

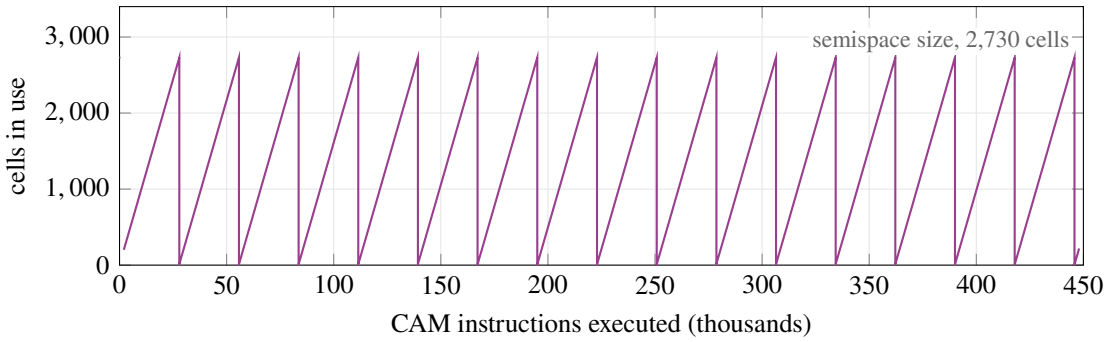


Figure 10: Heap occupancy while computing *fib*(20) on the CAM. Each drop is a garbage collection. The data come from replaying the compiled code with the implementation's allocation policy, which reproduces its count of 16 collections.

```
        LW      r1, [r8+2]              ; the closure [C : s]
        ...                             ; (check that r1 is a closure)
app_ok: MOV     r0, r10                 ; term = (s, t)
        ADDI    r10, CELL
        LI      r2, T_PAIR
        SW      r2, [r0]
        LW      r2, [r1+4]
        SW      r2, [r0+2]
        LW      r2, [r8+4]
        SW      r2, [r0+4]
        MOV     r8, r0
        PUSH    r9                      ; the return address
        LW      r9, [r1+2]              ; jump to C
        JMP     next
```

The return address is a code address pushed on the CAM stack, exactly as in the APP rule, and RET is `POP r9`.

8.3 Garbage collection

The heap is managed by Cheney's copying collector. Allocation is a pointer increment in the current semispace. When it is full, everything reachable is copied to the other semispace, and the two swap roles. The collector needs to know the *roots*, the places outside the heap that may point into it. In the CAM there are just two: the term and the stack. Two design decisions keep this simple.

- **Telling pointers apart.** The stack holds heap pointers, numbers and return addresses side by side. Numbers are odd, heap cells are at even addresses, and return addresses point into the code area, below the heap. So the collector can tell them apart by looking at the word alone, and no extra tags are needed.
- **Collecting only between instructions.** Every instruction that allocates first calls `reserve`, which collects if fewer than two cells are free. At that moment, everything live is reachable from the term and the stack, and no heap pointer is hidden in a VM16 register.

Figure 10 shows the heap during the computation of *fib*(20), which allocates two cells per call (one in CONS and one in APP), 43,784 in all. The heap fills, is collected, and fills again, 16 times. Hardly anything survives a collection: at most 16 cells, because the only live data are the environments of the calls in progress. This is the best case for a copying collector, whose cost depends on what survives rather than on what was allocated.

9 Programs with definitions

Writing a whole program as one term, with single-letter variables, soon becomes tiresome. The companion graph reducer reads programs as a series of named definitions followed by an expression, with long names, infix operators and `if... then... else`:

```
square x = x * x;
sumsq n = if n == 0 then 0 else square n + sumsq (n-1);
square (sumsq 3)
```

The CAM can run the same programs. A small Python program, `CAM/lam2cam.py`, translates them into the single term that `cam.asm` reads, and `CAM/run` translates a program and runs it:

```
$ python3 lam2cam.py sumsq.lam
# square = a
(\a.a ((rec b.\c.if (= c 0) 0 (+ (a c) (b (- c 1)))) 3)) (\a.* a a)
$ ./run sumsq.lam
196
[134 instructions, 0 collections]
```

Nothing in the machine or its compiler changes: the translation happens before the CAM sees the program, and it uses only constructions the CAM already has. It takes four steps.

1. Order the definitions. Starting from the final expression, the translator collects the definitions it uses, then the ones those use, and so on; definitions that are never used are left out. It orders them so that each comes after the definitions it refers to. Here *square* comes before *sumsq*.

2. Bind each definition with an application. A definition $f = F$ in front of an expression M means **let** $f = F$ **in** M, and that is the application $(\lambda f.\, M)\ F$. The CAM evaluates F once, pairs its value with the environment, and runs M there, so inside M the definition is an ordinary variable. Each definition is bound around everything that follows it, so the first definition ends up outermost.

3. Use rec for a definition that calls itself. If f occurs in its own body F, the binding is $(\lambda f.\, M)$ (**rec** $f.\, F$), and the REC instruction of section 6 builds the closure that refers to itself. This only works for functions: a recursive value such as an infinite list has no closure to refer to, and the translator rejects it.

4. Put a function used once in place of its use. Binding costs a few instructions and makes every variable access one step longer, so a definition that is a function and is used exactly once is substituted at its use instead. In the example, *sumsq* is used once and appears in place, as a **rec**; *square* is used twice and is bound. A program with a single definition, such as `CAM/fac.lam`,

```
fac n = if (= n 0) 1 (* n (fac (- n 1)));
fac 7
```

becomes exactly the term of section 6, and compiles to the same code: `(rec a.\b.if (= b 0) 1 (* b (a (- b 1)))) 7`.

Names. The reader of `cam.asm` only knows single letters, and it does not distinguish upper and lower case. The translator therefore names every variable after the depth of its binder: the outermost is `a`, the next `b`, and so on. Two variables in scope at the same time never share a name, so no renaming can capture a variable, and at most 26 binders may be nested. The letter counts binders from the outside, while the de Bruijn index that the compiler computes counts them from the inside. Inside *sumsq* above, `c` is ACC 0, `b` (*sumsq* itself) is ACC 1, and `a` (*square*) is ACC 2.

program	machine	machine steps	collections	VM16 instructions
fac 7, integers	SECD, with Z	322 transitions	0	29,738
	CAM, with Z	346 instructions	0	17,595
	CAM, with **rec**	171 instructions	0	10,131
fac 5, Church numerals	SECD	52,044 transitions	49	4,086,035
	CAM	73,681 instructions	10	1,505,151
fib(20)	SECD, with Z	—	702	61,161,761
	CAM, with **rec**	448,762 instructions	16	7,035,791
	VM16 assembly	—	—	175,169

Table 3: The same programs on the SECD machine, the CAM, and in hand-written VM16 code. The VM16 counts include reading and compiling the program, which is under 0.1 % of each run.

Lists. The language has no lists, but they can be written as functions, with the *Scott encoding*:

$$cons\ h\ t = \lambda c\, n.\, c\ h\ t \qquad\qquad nil = \lambda c\, n.\, n.$$

A list is taken apart by applying it to two things: a function of the head and the tail, for a non-empty list, and a value for the empty one. Each constructor of a data type becomes one argument, and the value passes its fields to it. The encoding is named after Dana Scott, who described it for numerals in unpublished lectures in the 1960s, as reported by Curry, Hindley and Seldin [6]. The better known Church encoding makes a list its own fold instead, $cons\ h\ t = \lambda c\, n.\, c\ h\ (t\ c\ n)$, which makes taking the tail expensive; Jansen compares the two [10], and Mogensen uses Scott's to represent λ-terms [15]. `CAM/list.lam` sums the primes below 100 with a sieve:

```
cons h t = \c n. c h t;
nil = \c n. n;

upto a b = if a > b then nil else cons a (upto (a+1) b);
filter p l = l (\h t. if p h then cons h (filter p t) else filter p t) nil;
sieve l = l (\h t. cons h (sieve (filter (\x. x % h != 0) t))) nil;
sum l = l (\h t. h + sum t) 0;

sum (sieve (upto 2 99))
```

It prints 1060 after 38,097 instructions and 4 collections. A cons cell is a closure here, and the pair the CAM uses for its environments would be cheaper (exercise 8); but nothing in the machine has to change.

What does not translate. The syntax is shared with the graph reducer, but the CAM evaluates strictly, and it has one form of recursion.

- Definitions that call each other, such as *even* and *odd*, are rejected: REC builds one closure that refers to itself, not several that refer to each other. Exercise 10 asks for an instruction that does.
- Programs that rely on laziness translate, but do not finish. A program that filters an infinite list of numbers builds the list before filtering it, and runs out of memory; one that uses the fixpoint combinator Y instead of Z runs out of stack.

10 Measurements

How fast is the CAM? The VM16 C interpreter counts the VM16 instructions it executes, which makes a fair, machine-independent measure. Table 3 compares the CAM with the SECD machine of the companion tutorial, implemented on VM16 in the same style, and with *fib*(20) written directly in VM16 assembly.

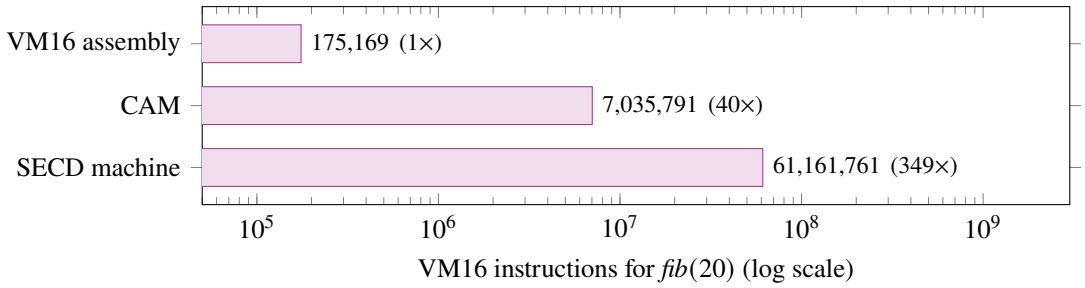


Figure 11: The cost of *fib*(20). The CAM is about 9 times faster than the interpreted SECD machine and about 40 times slower than hand-written code.

instruction	count	share	instruction	count	share
PUSH	98,508	22.0 %	BRANCH	21,891	4.9 %
SWAP	76,617	17.1 %	RET	21,891	4.9 %
ACC	76,617	17.1 %	LT	21,891	4.9 %
QUOTE	43,782	9.8 %	SUB	21,890	4.9 %
CONS	21,891	4.9 %	JUMP	10,946	2.4 %
APP	21,891	4.9 %	ADD	10,945	2.4 %

Table 4: The CAM instructions executed for *fib*(20), 448,762 in all (plus one REC and one STOP). Each of the 21,891 calls runs every instruction of the function body except one branch.

CAM against SECD. On *fib*(20) the CAM needs about a ninth of the work (fig. 11). The reasons are the three changes of section 2. Variables cost a fixed path of FSTs instead of a search by name; a call allocates one pair instead of control-list cells and a three-cell dump frame; and the SECD implementation also allocates a heap cell for every number, where the CAM's numbers are tagged words. With fewer allocations come fewer collections: 16 against 702.

CAM against hand-written code. The hand-written version keeps n in a register and allocates nothing; each call costs 8 VM16 instructions. The CAM spends 20.5 instructions per call of *fib*, each costing about 16 VM16 instructions, where they go as follows:

- **Dispatch.** Fetching and dispatching each CAM instruction, plus the jump back, takes 10 VM16 instructions, about 4.5 of the 7.0 million. A compiler that emitted VM16 code instead of CAM code would remove this.
- **Environment traffic.** PUSH, SWAP and ACC make up 56 % of the CAM instructions executed (table 4). They exist to save and restore environments, which hand-written code keeps in registers.
- **Allocation.** Two cells per call, plus the occasional collection.

On a 4 MHz Z80, where the VM16 interpreter averages about 250 clock cycles per VM16 instruction, these figures mean roughly 0.7 seconds for *fac* 7 with **rec** on the CAM and 2.2 seconds on the SECD machine, both measured on a Z80 emulator, and an estimated 7 minutes for *fib*(20) on the CAM.

11 Variations and descendants

Caml.
The first Caml compiled to the CAM. The machine's main weakness showed up there: a curried function of n arguments builds $n-1$ intermediate closures and environments, as $K\,4\,3$ did in table 2. Leroy's ZINC machine [14], the basis of Caml Light and of OCaml's bytecode interpreter, passes several arguments at once on a stack and avoids most of them, and it is the subject of an instructive report that compares its design with the CAM.

Compiling the CAM further.
Because CAM code is a simple sequence of instructions, it can be translated instruction by instruction into native code, removing the dispatch overhead measured in section 10.

Lazy evaluation.
The CAM is call-by-value, but closures can also represent suspended computations. Mauny and others studied lazy variants in which arguments are passed as closures and evaluated when needed.

Explicit substitutions.
The $\lambda\sigma$-calculus of Abadi, Cardelli, Curien and Lévy [1] grew out of the categorical combinators. It makes environments part of the calculus and explains the CAM, the SECD machine and Krivine's machine [11] as different strategies for carrying out substitutions.

Derivation.
Danvy and colleagues showed that the CAM, like the SECD machine, can be derived mechanically from an ordinary evaluator, by standard program transformations [2]. Douence and Fradet [8] describe the CAM, the SECD machine and other implementations in one framework, as combinations of a few independent design choices.

12 Summary

The CAM evaluates call-by-value λ-calculus with three registers. It compiles variables to *positions*, represents environments as *nested pairs* so that they are ordinary values, and runs *compiled code* whose return addresses live on the one stack. A handful of instructions suffices: ACC, QUOTE, PUSH, SWAP, CONS, CUR, APP and RET, plus a conditional branch and a recursive closure for practical programming. Read as functions, those instructions are the combinators of a cartesian closed category, which is where the machine's name comes from. In practice they make for a machine that is easy to implement, that needs only the term and the stack as garbage-collection roots, and that in our measurements is about nine times faster than the interpreted SECD machine.

Exercises

Exercise 1. Compile $(\lambda f\, x.\, f\, (f\, x))\ (+\ 1)\ 0$ by hand. How many instructions does it run? Check your answer with `python3 camdis.py` and with the instruction count that `cam.v16` prints (45).

Exercise 2. Translate $\lambda a.\lambda b.\, b\, a$ into categorical combinators, using the equations for $[\![x_i]\!]$, $[\![\lambda x.M]\!]$ and $[\![M\ N]\!]$. Then use the laws of section 7 to show that applying it to two values u and v gives $v\, u$.

Exercise 3. Complete the proof of theorem 1 for the rules of **if** and **rec**. What goes wrong if BRANCH does not pop the environment?

Exercise 4. The sequence PUSH; ACC i; SWAP appears whenever a variable is an operand. Design a single instruction that replaces it, describe its transition, and estimate from table 4 how many instructions it would save in $fib(20)$.

Exercise 5. Every call in the CAM pushes a return address, even a call in tail position such as $f\ (n-1)\ (a+n)$ in an accumulating loop. Design a tail-call instruction (compare TAP in the compiled SECD machine), show which sequence the compiler should replace with it, and explain why the stack then stays bounded for `(rec f.\n a.if (= n 0) a (f (- n 1) (+ a n))) 3000 0`.

Exercise 6. Variable access costs $i+1$ steps for index i. Describe an alternative representation where every variable costs one step, such as copying the values a closure needs into a flat vector when it is created. What does closure creation cost then? Which choice did Leroy make in ZINC?

Exercise 7. In `cam.asm`, = and < return 1 or 0, and **if** only accepts numbers. Change them to work with Church booleans $\lambda x\, y.x$ and $\lambda x\, y.y$. How can BRANCH recognise them?

Exercise 8. Add pairs to the source language: $\langle M, N\rangle$, *fst* M and *snd* M. Show that they compile to instructions the CAM already has, and that the categorical law $\langle f, g\rangle$; Fst $= f$ then justifies an optimisation.

Exercise 9. In fig. 10, at most 16 cells survive a collection. Explain where this bound comes from, in terms of the depth of the recursion and the cells that a pending call of *fib* keeps alive.

Exercise 10. `lam2cam.py` rejects mutually recursive definitions such as *even* n = **if** $n = 0$ **then** 1 **else** *odd* $(n-1)$ and its partner *odd*. Design an instruction REC2 C_1, C_2 that builds two closures whose environments both contain both closures, in the style of fig. 7. Which indices do *even* and *odd* have inside their bodies?

Exercise 11. Write a CAM interpreter in a language of your choice from the reference card alone, and test it on the examples of sections 5 and 6.

13 Reference card

Machine	(start $\langle (), P, \varepsilon\rangle$; halt at STOP, with the result in the term)
ACC i	$\langle t, \mathsf{ACC}\, i\, ; C, S\rangle \to \langle snd(fst^i(t)), C, S\rangle$
QUOTE n	$\langle t, \mathsf{QUOTE}\, n\, ; C, S\rangle \to \langle n, C, S\rangle$
PUSH	$\langle t, \mathsf{PUSH}\, ; C, S\rangle \to \langle t, C, t.S\rangle$
SWAP	$\langle t, \mathsf{SWAP}\, ; C, s.S\rangle \to \langle s, C, t.S\rangle$
CONS	$\langle t, \mathsf{CONS}\, ; C, s.S\rangle \to \langle (s, t), C, S\rangle$
CUR C'	$\langle t, \mathsf{CUR}\, C'\, ; C, S\rangle \to \langle [C' : t], C, S\rangle$
APP	$\langle ([C' : s], t), \mathsf{APP}\, ; C, S\rangle \to \langle (s, t), C', C.S\rangle$
RET	$\langle t, \mathsf{RET}\, ; C, C'.S\rangle \to \langle t, C', S\rangle$
op	$\langle n_2, op\, ; C, n_1.S\rangle \to \langle n_1\ op\ n_2, C, S\rangle$
BRANCH	$\langle b, \mathsf{BRANCH}\, C_1\, C_2\, ; C, s.S\rangle \to \langle s, C_b\, ; C, S\rangle$
REC C'	$\langle t, \mathsf{REC}\, C'\, ; C, S\rangle \to \langle k, C, S\rangle,\ k = [C' : (t, k)]$

Compiler	(ρ = names in scope, innermost first)
variable	$[\![x]\!]_\rho$ = ACC (position of x in ρ)
constant	$[\![n]\!]_\rho$ = QUOTE n
abstraction	$[\![\lambda x.M]\!]_\rho$ = CUR ($[\![M]\!]_{x:\rho}$; RET)
application	$[\![M\ N]\!]_\rho$ = PUSH;$[\![M]\!]_\rho$;SWAP;$[\![N]\!]_\rho$;CONS;APP
operator	$[\![op\ M\ N]\!]_\rho$ = PUSH;$[\![M]\!]_\rho$;SWAP;$[\![N]\!]_\rho$; op
conditional	$[\![\mathbf{if}\ c\ t\ e]\!]_\rho$ = PUSH;$[\![c]\!]_\rho$;BRANCH $[\![t]\!]_\rho$ $[\![e]\!]_\rho$
recursion	$[\![\mathbf{rec}\ f.\lambda x.M]\!]_\rho$ = REC ($[\![M]\!]_{x:f:\rho}$; RET)

Combinators

$[\![x_i]\!]$ = Fst^i ; Snd $[\![\lambda x.M]\!] = \Lambda([\![M]\!])$ $[\![M\ N]\!] = \langle [\![M]\!], [\![N]\!]\rangle$; App

$\langle f, g\rangle$; Fst $= f$ $\langle f, g\rangle$; Snd $= g$ $\langle h\, ; \Lambda(f), g\rangle$; App $= \langle h, g\rangle\, ; f$

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Lévy. Explicit substitutions. *Journal of Functional Programming*, 1(4):375–416, 1991.

[2] M. S. Ager, D. Biernacki, O. Danvy, and J. Midtgaard. From interpreter to compiler and virtual machine: a functional derivation. Technical Report RS-03-14, BRICS, University of Aarhus, 2003.

[3] G. Cousineau, P.-L. Curien, and M. Mauny. The categorical abstract machine. In *Functional Programming Languages and Computer Architecture (FPCA)*, volume 201 of *LNCS*, pages 50–64. Springer, 1985.

[4] G. Cousineau, P.-L. Curien, and M. Mauny. The categorical abstract machine. *Science of Computer Programming*, 8(2):173–202, 1987.

[5] P.-L. Curien. *Categorical Combinators, Sequential Algorithms and Functional Programming*. Pitman, 1986. Second edition, Birkhäuser, 1993.

[6] H. B. Curry, J. R. Hindley, and J. P. Seldin. *Combinatory Logic*, volume II. North-Holland, 1972.

[7] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae*, 34:381–392, 1972.

[8] R. Douence and P. Fradet. A systematic study of functional language implementations. *ACM Transactions on Programming Languages and Systems*, 20(2):344–387, 1998.

[9] P. Henderson. *Functional Programming: Application and Implementation*. Prentice-Hall, 1980.

[10] J. M. Jansen. Programming in the λ-calculus: from Church to Scott and back. In *The Beauty of Functional Code*, volume 8106 of *LNCS*. Springer, 2013.

[11] J.-L. Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic Computation*, 20(3):199–207, 2007.

[12] J. Lambek and P. J. Scott. *Introduction to Higher Order Categorical Logic*. Cambridge University Press, 1986.

[13] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.

[14] X. Leroy. The ZINC experiment: an economical implementation of the ML language. Technical Report 117, INRIA, 1990. https://caml.inria.fr/pub/papers/xleroy-zinc.pdf

[15] T. Æ. Mogensen. Efficient self-interpretation in lambda calculus. *Journal of Functional Programming*, 2(3):345–364, 1992.